School of Computer Science and Artificial Intelligence

Lab Assignment # 21.5

Task

Program : B. Tech (CSE)
Specialization :AIML
Course Title : AI Assisted Coding
Course Code : 23CS002PC304
Semester : VI
Academic Session : 2025-2026
Name of Student :K.Srija
Enrollment No. : 2303A52189
Batch No. : 34
Date :14/04/26

A: AI-Based Initial Program Development

Prompt:

Generate a basic Python program for a Student Grade Calculator that takes marks as input and calculates the grade. Enhance the program by adding input validation and additional features like average calculation and grade classification.

Code:

Initial Code (Basic Version)

```python
# Basic Student Grade Calculator

marks = int(input("Enter marks: "))

if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 50:
    print("Grade: C")
else:
    print("Grade: Fail")
```

Enhanced Code (Improved Version):

```python
# Enhanced Student Grade Calculator

marks_list = []

n = int(input("Enter number of subjects: "))

for i in range(n):
    marks = float(input(f"Enter marks for subject {i+1}: "))

    # Validation
    if marks < 0 or marks > 100:
        print("❌ Invalid marks! Enter between 0-100")
        exit()

    marks_list.append(marks)

average = sum(marks_list) / n

# Grade classification
if average >= 90:
    grade = "A"
elif average >= 75:
    grade = "B"
elif average >= 50:
    grade = "C"
else:
    grade = "Fail"

print("\nAverage Marks:", average)
print("Final Grade:", grade)
```

```
Enter marks: 78
Grade: B
```

Task

Sample Input/Output:

Initial Code (Basic Version)

Enhanced Code (Improved Version)

```
Enter number of subjects: 6
Enter marks for subject 1: 67
Enter marks for subject 2: 80
Enter marks for subject 3: 90
Enter marks for subject 4: 95
Enter marks for subject 5: 87
Enter marks for subject 6: 75

Average Marks: 82.33333333333333
Final Grade: B
```

Code Explanation:

1. This task demonstrates AI-assisted program development and enhancement.
2. The initial version calculates grade based on a single input mark.
3. It lacks validation and does not handle multiple subjects.
4. The enhanced version allows multiple inputs for different subjects.
5. Input validation ensures marks are within valid range (0–100).
6. The program calculates average marks for accurate grading.
7. Additional features improve usability and correctness.
8. This shows how AI can refine and optimize code effectively.

B: AI-Assisted Application Improvement

Prompt:

Generate a basic Python program for a Library Management System with simple book storage functionality. Enhance the program by adding features like search, issue, return, and improved structure.

Code:

Initial Code (Basic Version)

```python
# Basic Library Management

books = ["Python", "Java", "C++"]

print("Available Books:")
for book in books:
    print(book)
```

Enhanced Code (Improved Version)

Task

```python
# Enhanced Library Management System

library = ["Python", "Java", "C++", "AI"]
issued_books = []

def display_books():
    print("\nAvailable Books:")
    for book in library:
        print(book)

def search_book(name):
    if name in library:
        print("✅ Book found")
    else:
        print("❌ Book not available")

def issue_book(name):
    if name in library:
        library.remove(name)
        issued_books.append(name)
        print("✅ Book issued")
    else:
        print("❌ Book not available")

def return_book(name):
    if name in issued_books:
        issued_books.remove(name)
        library.append(name)
        print("✅ Book returned")
    else:
        print("❌ Invalid return")

# Demo Execution
display_books()
search_book("Python")
issue_book("Python")
display_books()
return_book("Python")
display_books()
```

<u>Sample Input/Output</u>:

Initial Code (Basic Version)

```
Available Books:
Python
Java
C++
```

Enhanced Code (Improved Version)

```
Available Books:
Python
Java
C++
AI
✅ Book found
✅ Book issued

Available Books:
Java
C++
AI
✅ Book returned

Available Books:
Java
C++
AI
Python
```

Code Explanation:

1. This task demonstrates improving an AI-generated application.
2. The basic version only displays available books.
3. It lacks functionality for real-world usage.
4. The enhanced version introduces multiple features like search, issue, and return.
5. Functions are used to improve modularity and readability.
6. Book availability is dynamically updated during operations.
7. Error handling ensures proper responses for invalid actions.
8. This shows how AI can enhance both structure and functionality.

C : Git Workflow Practice

Prompt:

Demonstrate how to initialize a Git repository, create a new branch, and add AI-generated changes. Perform commit and merge operations to integrate changes into the main branch.

CodeSample Input/Output:

```
shish@KSB MINGW64 ~/M (master)
$ git init
Initialized empty Git repository in C:/Users/shish/M/.git/
```

Task

```
shish@KSB MINGW64 ~/M (master)
$ git add index.html
```

```
shish@KSB MINGW64 ~/M (master)
$ git commit -m "Intial commit: Added Portfolio page"
[master (root-commit) fc355d3] Intial commit: Added Portfolio page
 1 file changed, 9 insertions(+)
 create mode 100644 index.html
```

```
shish@KSB MINGW64 ~/M (master)
$ git remote add origin https://github.com/Kodambhagath/MPortfolio.git

shish@KSB MINGW64 ~/M (master)
$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 371 bytes | 371.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Kodambhagath/MPortfolio.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

shish@KSB MINGW64 ~/M (master)
$ |
```

<u>Code Explanation</u>:

1. This task demonstrates integration of AI-generated code with Git version control.
2. The repository is initialized using git init.
3. Initial code is added and committed to the main branch.
4. A new branch is created to work on AI-generated improvements.
5. Changes are committed separately in the feature branch.
6. The main branch is then updated by merging the feature branch.
7. This ensures safe and organized code development.
8. Git helps track changes and supports collaborative workflows.

Task D: Branching and Merging with AI Support

Prompt:

Demonstrate branching and merging in Git by developing a program in the main branch and enhancing it in a feature branch using AI. Merge the changes back into the main branch and verify the final output.

Code:

```
> git init
Initialized empty Git repository in /home/user/my-project/.git/

> touch calculator.py
> nano calculator.py
# added initial calculator code
```

```
> git add calculator.py
> git commit -m "Added basic calculator program"
[master (root-commit) 5a3f29d] Added basic calculator program
 1 file changed, 10 insertions(+)
 create mode 100644 calculator.py
```

```
> git checkout -b feature-enhancement
Switched to a new branch 'feature-enhancement'

> # (modify calculator.py to add features)
> git add calculator.py
> git commit -m "Enhanced calculator with more operations using AI"
[feature-enhancement a1b2c3d] Enhanced calculator with more operations using AI
 1 file changed, 25 insertions(+), 5 deletions(-)
```

```
> git checkout master
Switched to branch 'master'
> git merge feature-enhancement
Merge made by the 'recursive' strategy.
 calculator.py | 30 ++++++++++++++++++++++++++++-----
 1 file changed, 25 insertions(+), 5 deletions(-)
```

```
> python3 calculator.py
# (output of the calculator program demonstrating new features)
Select operation:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice (1/2/3/4): 3
Enter first number: 5
Enter second number: 4
5 * 4 = 20
```

Sample Input/Output:

```
[main] Added basic calculator program
Switched to branch 'feature-enhancement'
[feature-enhancement] Enhanced calculator with more operations using AI
Switched to branch 'main'
Updating...
Merge made by the 'ort' strategy
```

Code Explanation:

1. This task demonstrates branching and merging with AI-assisted development.
2. The main branch contains the initial version of the program.
3. A feature branch is created to add enhancements using AI.
4. Changes are committed separately in the feature branch.
5. The main branch remains stable during development.
6. After completion, changes are merged back into the main branch.
7. The final program is tested to verify correct functionality.
8. This workflow supports safe and efficient collaborative development.

Task E: AI for Commit Message Writing

Prompt:

Generate meaningful Git commit messages for multiple code changes using AI and compare them with manually written messages. Ensure the messages clearly describe the changes and follow best practices.

Code:

```
# Manual commit message git
commit -m "Updated code"
```

```
# AI-generated commit message git commit
-m "Added input validation and improved
calculator functionality"
```

Sample Input/Output:

```
Manual Message:
"Updated code"

AI Message:
"Added input validation, division feature, and improved user interface for better usability"
```

Code Explanation:

1. This task focuses on improving commit message quality using AI.
2. Manual commit messages are often short and unclear.
3. Poor messages make it difficult to track changes in a project.
4. AI-generated messages are more descriptive and structured.
5. They clearly explain what changes were made and why.
6. This improves collaboration and version tracking.
7. Better commit messages help in debugging and maintenance.
8. AI assists developers in following best practices.

Task

E: Pair Programming with AI Assistance Prompt:

Simulate pair programming where one developer writes initial code and another uses AI to enhance and optimize it. Integrate both contributions using Git and verify the final improved program.

Code:

Initial Code (Student A)

```
# Basic calculator (Student A)

num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

print("Sum:", num1 + num2)
```

Enhanced Code (Student B using AI):

Task

```python
# Enhanced calculator (Student B with AI)

try:
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))

    print("\nOperations:")
    print("1. Add\n2. Subtract\n3. Multiply\n4. Divide")

    choice = int(input("Enter choice: "))

    if choice == 1:
        print("Result:", num1 + num2)
    elif choice == 2:
        print("Result:", num1 - num2)
    elif choice == 3:
        print("Result:", num1 * num2)
    elif choice == 4:
        if num2 == 0:
            print("❌ Cannot divide by zero")
        else:
            print("Result:", num1 / num2)
    else:
        print("❌ Invalid choice")

except ValueError:
    print("❌ Enter valid numbers")
```

<u>Sample Input/Output</u>:

Initial Code (Student A)

```
Enter first number: 4
Enter second number: 2
Sum: 6
```

Enhanced Code (Student B using AI)

```
Enter first number: 7
Enter second number: 3

Operations:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice: 2
Result: 4.0
```

Code Explanation:

1. This task demonstrates pair programming with AI assistance.
2. Student A develops a basic calculator with limited functionality.
3. Student B uses AI to enhance the program with more features.
4. Additional operations like subtraction, multiplication, and division are added.
5. Exception handling improves reliability and prevents crashes.
6. User interaction is enhanced with a menu-driven approach.
7. Both contributions are integrated using Git workflow.
8. This shows how AI supports collaborative development effectively.

G: Collaborative Development using Branches

Prompt:

Simulate collaborative development where two developers work on different features using Git branches and AI assistance. Merge both contributions into a single project and verify the final integrated output.

Code:

(Feature Implementation by Two Students) ○ Student A (Feature 1 — Addition & Subtraction)

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

Student B (Feature 2 — Multiplication & Division using AI)

Task

```python
def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "❌ Cannot divide by zero"
    return a / b
```

Final Integrated Code (After Merge)

```python
# Combined Calculator

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "❌ Cannot divide by zero"
    return a / b

# Input
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

print("Add:", add(a, b))
print("Subtract:", subtract(a, b))
print("Multiply:", multiply(a, b))
print("Divide:", divide(a, b))
```

<u>Sample Input/Output</u>:

Final Integrated Code (After Merge)

```
Enter first number: 6
Enter second number: 4
Add: 10.0
Subtract: 2.0
Multiply: 24.0
Divide: 1.5
```

<u>Code Explanation</u>:

1. This task demonstrates collaborative development using Git branches.
2. Student A and Student B work on different features independently.
3. Student A implements basic operations like addition and subtraction.
4. Student B uses AI to implement advanced operations like multiplication and division.
5. Each feature is developed in separate branches.
6. The features are merged into a single integrated program.
7. The final program combines all functionalities and works correctly.
8. This approach improves teamwork and efficient development.

H: AI-Supported Team Coding Workflow

Prompt:

Simulate a team coding workflow where multiple developers collaborate using AI for suggestions and improvements. Manage code integration using Git branching and merging to produce a final working application.

Code:

```
Enter first number: 4
Enter second number: 6

Operations:
1.Add  2.Subtract  3.Multiply  4.Divide  5.Power
Enter choice: 2
Result: -2.0
```

Task

```python
# Team Project: Smart Calculator with AI Enhancements

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "❌ Cannot divide by zero"
    return a / b

# AI Enhancement: Power function
def power(a, b):
    return a ** b

try:
    a = float(input("Enter first number: "))
    b = float(input("Enter second number: "))

    print("\nOperations:")
    print("1.Add  2.Subtract  3.Multiply  4.Divide  5.Power")

    choice = int(input("Enter choice: "))

    if choice == 1:
        print("Result:", add(a, b))
    elif choice == 2:
        print("Result:", subtract(a, b))
    elif choice == 3:
        print("Result:", multiply(a, b))
    elif choice == 4:
        print("Result:", divide(a, b))
    elif choice == 5:
        print("Result:", power(a, b))
    else:
        print("❌ Invalid choice")

except ValueError:
    print("❌ Enter valid numbers")
```

<u>Sample Input/Output</u>:

<u>Code Explanation</u>:

1. This task simulates a real-world team coding workflow with AI support.
2. Multiple developers contribute different features to the project.
3. Basic operations are implemented collaboratively across branches.
4. AI is used to suggest and add new functionality like the power operation.
5. Git branching allows parallel development without conflicts.
6. Changes are merged into a final integrated application.

7. Exception handling ensures robustness and reliability.
8. This demonstrates efficient team collaboration using AI and version control.